



LeetCode 412: Fizz Buzz

1. Problem Title & Link

Title: LeetCode 412: Fizz Buzz

Link: <https://leetcode.com/problems/fizz-buzz/>

2. Problem Statement (Short Summary)

Given an integer n , generate a list of strings from 1 to n .

Rules:

- If a number is divisible by **3**, write "Fizz"
- If a number is divisible by **5**, write "Buzz"
- If a number is divisible by both **3 and 5**, write "FizzBuzz"
- Otherwise, write the number itself as a string

Return the resulting list.

3. Examples (Input → Output)

Example 1

Input:

$n = 3$

Output:

["1", "2", "Fizz"]

Explanation:

1 → "1"

2 → "2"

3 → "Fizz"

Example 2

Input:

$n = 5$

Output:

["1", "2", "Fizz", "4", "Buzz"]

Example 3

Input:

$n = 15$

Output:

["1", "2", "Fizz", "4", "Buzz",
"Fizz", "7", "8", "Fizz", "Buzz",



```
"11","Fizz","13","14","FizzBuzz"]
```

4. Constraints

$1 \leq n \leq 10^4$

Requirements:

Return a list of strings.

Maintain order from 1 to n.

5. Core Concept (Pattern / Topic)

Simulation / Conditional Logic

Secondary Topics:

- Modulo Operator (%)
- Looping
- String Handling

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

For every number from:

$1 \rightarrow n$

Check:

Divisible by 3?

Divisible by 5?

Divisible by both?

and append the correct answer.

Important Observation

Many students write:

if $i \% 3 == 0$:

 "Fizz"

elif $i \% 5 == 0$:

 "Buzz"

elif $i \% 15 == 0$:

 "FizzBuzz"

This is WRONG.

Why?

For:

$i = 15$



First condition becomes true:

$15 \% 3 == 0$

So "Fizz" gets printed.

The "FizzBuzz" condition is never reached.

Correct Order

Always check:

Divisible by BOTH first

Then 3

Then 5

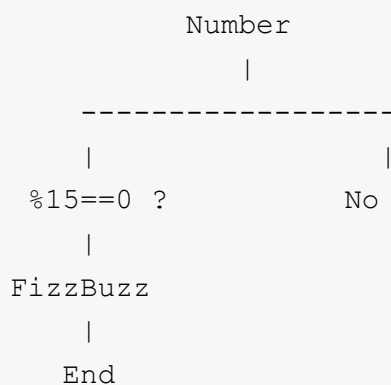
7. Visual / Intuition Diagram (ASCII Diagram)

```

1 → "1"
2 → "2"
3 → "Fizz"
4 → "4"
5 → "Buzz"
6 → "Fizz"
7 → "7"
8 → "8"
9 → "Fizz"
10 → "Buzz"
11 → "11"
12 → "Fizz"
13 → "13"
14 → "14"
15 → "FizzBuzz"

```

Decision Tree:



Else

```

      |
      %3==0 ?
      |

```



```
Fizz

Else
    |
    %5==0 ?
    |
    Buzz

Else
    Number
```

8. Pseudocode (Language Independent)

```
create result list

for i from 1 to n

    if i % 15 == 0
        add "FizzBuzz"

    else if i % 3 == 0
        add "Fizz"

    else if i % 5 == 0
        add "Buzz"

    else
        add string(i)

return result
```

9. Code Implementation

Python

```
class Solution:
    def fizzBuzz(self, n: int):

        result = []

        for i in range(1, n + 1):

            if i % 15 == 0:
                result.append("FizzBuzz")
```



```
        elif i % 3 == 0:
            result.append("Fizz")

        elif i % 5 == 0:
            result.append("Buzz")

        else:
            result.append(str(i))

    return result
```

Java

```
import java.util.*;

class Solution {

    public List<String> fizzBuzz(int n) {

        List<String> result = new ArrayList<>();

        for(int i = 1; i <= n; i++) {

            if(i % 15 == 0)
                result.add("FizzBuzz");

            else if(i % 3 == 0)
                result.add("Fizz");

            else if(i % 5 == 0)
                result.add("Buzz");

            else
                result.add(String.valueOf(i));

        }

        return result;

    }

}
```

10. Time & Space Complexity

Time Complexity



$O(n)$

Reason:

One iteration for each number.

Space Complexity

$O(n)$

Reason:

Result list stores n strings.

11. Common Mistakes / Edge Cases

Mistake 1

Wrong ordering:

```
if i % 3 == 0
```

```
elif i % 5 == 0
```

```
elif i % 15 == 0
```

For:

```
15
```

Output:

```
Fizz
```

Expected:

```
FizzBuzz
```

Mistake 2

Forgetting string conversion.

Wrong:

```
result.append(i)
```

Expected:

```
result.append(str(i))
```

Mistake 3

Loop range mistake.

Wrong:

```
range(n)
```

Starts from 0.

Need:

```
range(1, n + 1)
```



12. Detailed Dry Run (Step-by-Step Table)

Input:

n = 15

i	Div by 3	Div by 5	Output
1	No	No	"1"
2	No	No	"2"
3	Yes	No	"Fizz"
4	No	No	"4"
5	No	Yes	"Buzz"
6	Yes	No	"Fizz"
7	No	No	"7"
8	No	No	"8"
9	Yes	No	"Fizz"
10	No	Yes	"Buzz"
11	No	No	"11"
12	Yes	No	"Fizz"
13	No	No	"13"
14	No	No	"14"
15	Yes	Yes	"FizzBuzz"

Final Output:

```
[ "1", "2", "Fizz", "4", "Buzz",
  "Fizz", "7", "8", "Fizz", "Buzz",
  "11", "Fizz", "13", "14", "FizzBuzz" ]
```

13. Common Use Cases (Real-Life / Interview)

Although Fizz Buzz is simple, interviewers use it to test:

Basic Coding Ability

Loops

Conditions

Modulo

Strings

Following Requirements

Can the candidate implement rules correctly?

**Code Quality**

Readable code

Clean structure

Edge case handling

Many companies use Fizz Buzz as an initial screening question.

14. Common Traps (Important!)**Trap 1**

Checking 3 before 15.

Most common mistake.

Trap 2

Returning integers instead of strings.

Wrong:

```
[1,2,"Fizz"]
```

Correct:

```
["1","2","Fizz"]
```

Trap 3

Using multiple independent if statements.

Example:

```
if i % 3 == 0:
```

```
    result.append("Fizz")
```

```
if i % 5 == 0:
```

```
    result.append("Buzz")
```

For:

```
15
```

Output becomes:

```
["Fizz", "Buzz"]
```

instead of:

```
["FizzBuzz"]
```

15. Builds To (Related LeetCode Problems)

Progression:

LC 412 - Fizz Buzz





LC 202 - Happy Number



LC 728 - Self Dividing Numbers



LC 204 - Count Primes



LC 263 - Ugly Number

These strengthen number manipulation skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
If-Else Chain	$O(n)$	$O(1)$	Best
String Builder	$O(n)$	$O(n)$	Flexible
HashMap Rules	$O(1)$	$O(1)$	Useful for extensions

Alternative: Build String Dynamically

Instead of checking %15.

```
s = ""
```

```
if i % 3 == 0:
```

```
    s += "Fizz"
```

```
if i % 5 == 0:
```

```
    s += "Buzz"
```

```
if s == "":
```

```
    s = str(i)
```

```
For:
```

```
15
```

```
Output:
```

```
FizzBuzz
```

Automatically.

This is often preferred in follow-up interviews.

17. Why This Solution Works (Short Intuition)

Every number is checked against the required divisibility rules exactly once.



Checking %15 first ensures numbers divisible by both 3 and 5 correctly produce "FizzBuzz" instead of "Fizz" or "Buzz".

18. Variations / Follow-Up Questions

Follow-Up 1

Replace:

3 → Fizz

5 → Buzz

with:

7 → Pop

11 → Boom

Can your solution be generalized?

Follow-Up 2

Generate Fizz Buzz for a range:

100 to 200

instead of:

1 to n

Follow-Up 3

Print instead of returning.

How would complexity change?

Follow-Up 4

Create a configurable Fizz Buzz.

Example:

```
{  
  3: "Fizz",  
  5: "Buzz",  
  7: "Pop"  
}
```

Output:

105 → FizzBuzzPop

This is a common senior-level extension of the classic Fizz Buzz problem.